

```
!ls /content
```

```
sample_data
```

```
#Task 2 Requirement 1 Import the motif dataset
import pandas as pd
import glob
import os

data_dir = "/content"

all_files = glob.glob(os.path.join(data_dir, "*-clips-angles.csv"))
print("Files found:", len(all_files))
print(all_files[:5])

dfs = [pd.read_csv(f) for f in all_files]
data = pd.concat(dfs, ignore_index=True)

print("Combined dataset shape:", data.shape)
data.head()

print(data.shape)
data.head()
```

Files found: 25

['/content/2x4-clips-angles.csv', '/content/3x4-clips-angles.csv', '/content/1x1

Combined dataset shape: (201376, 87)

(201376, 87)

	Motifname	Motifseq	bb_alpha_mean	bb_alph
0	8WIF_GGCUGCCCUAAGCG_a1237_1	GGCUGCGCGAAUCC	-0.311286	
1	8WID_GGCUGCCCUAAGCG_a1237_1	GGCUGCGCGAAUCC	-0.334143	
2	8WIC_CAAACUGUCAUCGA_A1025_1	AGCUACUGCAAACU	-0.878500	
3	8WIB_GGCUGCCCUAAGCG_a1237_1	GGCUGCGCGAAUCC	-0.334286	
4	8WIB_CAAACUGUCAUCGA_A1025_1	AGCUACUGCAAACU	-0.878571	

5 rows × 87 columns

```
#Requirement 2 merge and convert labels to numeric
from sklearn.preprocessing import LabelEncoder
```

```
id_cols = ["Motifname", "Motifseq"]
X_df = data.drop(columns=id_cols + ["Label"])
```

```
le = LabelEncoder()
y = le.fit_transform(data["Label"])
```

```
print("Classes:", le.classes_)
```

```
Classes: ['1x1-clips' '1x2-clips' '1x3-clips' '1x4-clips' '1x5-clips' '2x2-clips'
'2x3-clips' '2x4-clips' '2x5-clips' '3x3-clips' '3x4-clips' '3x5-clips'
'4x4-clips' '4x5-clips' '5x5-clips' 'bulge1-clips' 'bulge2-clips'
'bulge3-clips' 'bulge4-clips' 'bulge5-clips' 'hairpin3-clips'
'hairpin4-clips' 'hairpin5-clips' 'hairpin6-clips' 'hairpin7-clips']
```

```
#Requirement 3 preprocess and save X and y
# Check missing values
X_df.isna().sum().sum()
```

```
# Fix missing values using median imputation
X_df = X_df.fillna(X_df.median())
```

```
print("Missing values after imputation:", X_df.isna().sum().sum())
```

Missing values after imputation: 0

We identified 2 missing values in the numeric feature matrix. Since all features represent continuous RNA structural descriptors, we applied median imputation to replace missing values. Median imputation is robust to outliers and preserves the distribution of the data. After imputation, the dataset contained zero missing values, ensuring a clean feature matrix for downstream machine learning tasks.

```
#requirement 4
# look at the dataset
data.head()
# Shape of the dataset
print("Dataset shape:", data.shape)

# Summary of columns and data types
data.info()
```

```
Dataset shape: (201376, 87)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 201376 entries, 0 to 201375
Data columns (total 87 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Motifname                             201376 non-null object
1   Motifseq                              201376 non-null object
2   bb_alpha_mean                         201376 non-null float64
3   bb_alpha_median                      201376 non-null float64
4   bb_alpha_std                         201376 non-null float64
5   bb_alpha_range                       201376 non-null float64
6   bb_alpha_skewness                    201376 non-null float64
7   bb_alpha_kurtosis                    201376 non-null float64
8   bb_beta_mean                         201376 non-null float64
9   bb_beta_median                      201376 non-null float64
10  bb_beta_std                         201376 non-null float64
11  bb_beta_range                       201376 non-null float64
12  bb_beta_skewness                    201376 non-null float64
13  bb_beta_kurtosis                    201376 non-null float64
14  bb_gamma_mean                       201376 non-null float64
15  bb_gamma_median                     201376 non-null float64
16  bb_gamma_std                       201376 non-null float64
17  bb_gamma_range                      201376 non-null float64
18  bb_gamma_skewness                   201376 non-null float64
19  bb_gamma_kurtosis                   201376 non-null float64
20  bb_delta_mean                       201376 non-null float64
21  bb_delta_median                     201376 non-null float64
```

```
22 bb_delta_std 201376 non-null float64
23 bb_delta_range 201376 non-null float64
24 bb_delta_skewness 201376 non-null float64
25 bb_delta_kurtosis 201376 non-null float64
26 bb_eps_mean 201376 non-null float64
27 bb_eps_median 201376 non-null float64
28 bb_eps_std 201376 non-null float64
29 bb_eps_range 201376 non-null float64
30 bb_eps_skewness 201376 non-null float64
31 bb_eps_kurtosis 201376 non-null float64
32 bb_zeta_mean 201376 non-null float64
33 bb_zeta_median 201376 non-null float64
34 bb_zeta_std 201376 non-null float64
35 bb_zeta_range 201376 non-null float64
36 bb_zeta_skewness 201376 non-null float64
37 bb_zeta_kurtosis 201376 non-null float64
38 bb_chi_mean 201376 non-null float64
39 bb_chi_median 201376 non-null float64
40 bb_chi_std 201376 non-null float64
41 bb_chi_range 201376 non-null float64
42 bb_chi_skewness 201376 non-null float64
43 bb_chi_kurtosis 201376 non-null float64
44 pucker_Phase_mean 201376 non-null float64
45 pucker_Phase_median 201376 non-null float64
46 pucker_Phase_std 201376 non-null float64
47 pucker_Phase_range 201376 non-null float64
48 pucker_Phase_skewness 201376 non-null float64
49 pucker_Phase_kurtosis 201376 non-null float64
50 pucker_amplitude_mean 201376 non-null float64
51 pucker_amplitude_median 201376 non-null float64
```

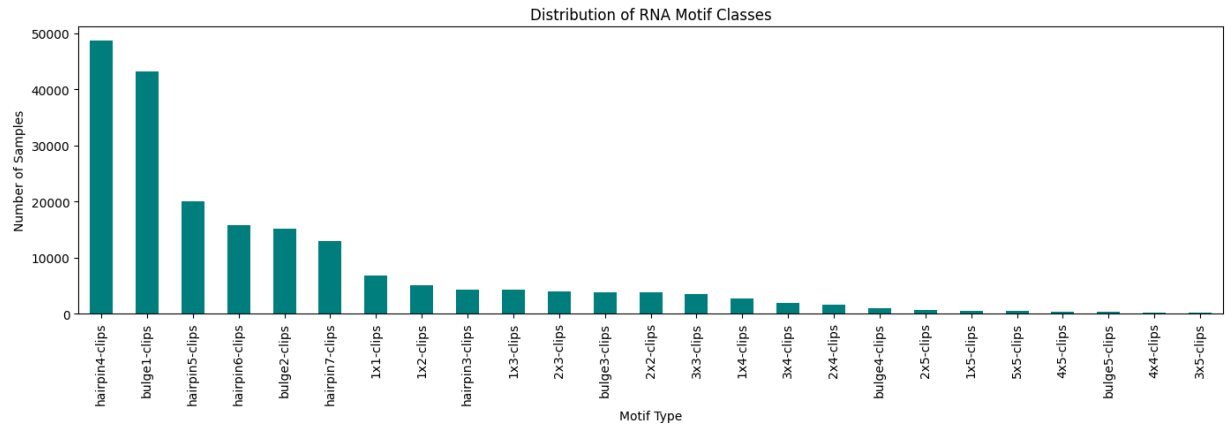
```
#summary stats
data.describe()
```

	bb_alpha_mean	bb_alpha_median	bb_alpha_std	bb_alpha_range	bb_alpha_s
count	201376.000000	201376.000000	201376.000000	201376.000000	201376
mean	-0.496659	-0.910466	1.350011	4.395770	0
std	0.494606	0.480681	0.355213	1.015213	0
min	-2.158375	-2.681000	0.300155	1.124000	-1
25%	-0.840111	-1.136500	1.142877	3.845000	0
50%	-0.534333	-1.043500	1.392461	4.412000	0
75%	-0.177000	-0.903000	1.597512	5.218000	1
max	2.344143	2.969000	2.795456	6.276000	2

8 rows × 84 columns

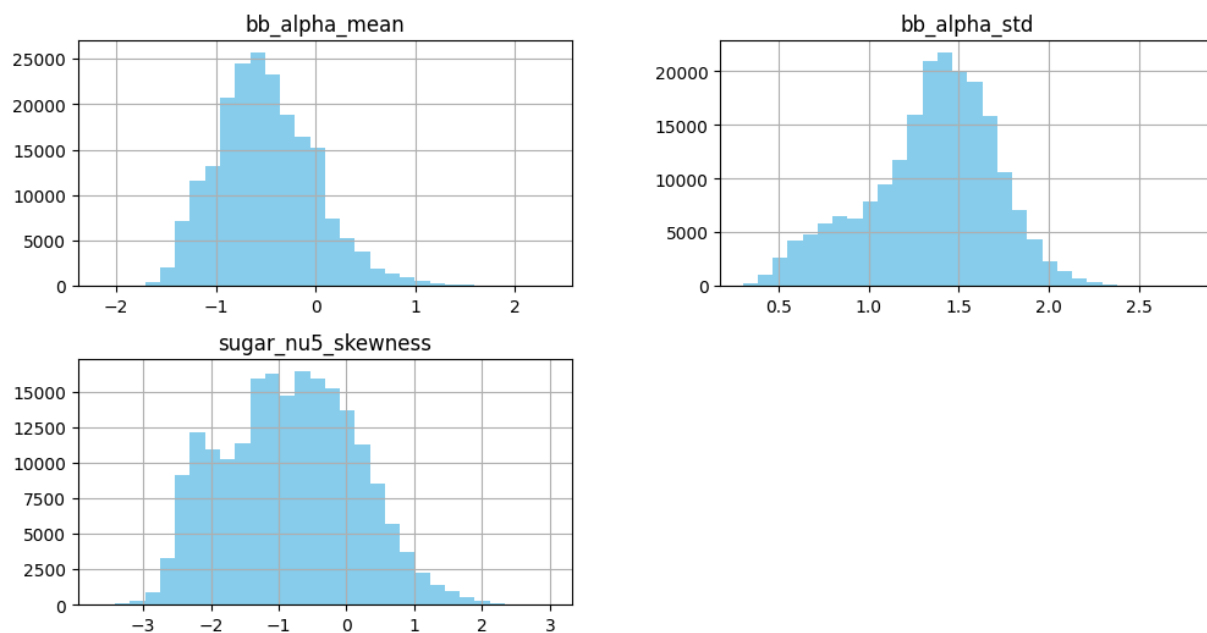
```
import matplotlib.pyplot as plt

plt.figure(figsize=(14,5))
data["Label"].value_counts().plot(kind="bar", color="teal")
plt.title("Distribution of RNA Motif Classes")
plt.xlabel("Motif Type")
plt.ylabel("Number of Samples")
plt.xticks(rotation=90)
plt.tight_layout()
plt.show()
```



```
#histogram
data[['bb_alpha_mean', 'bb_alpha_std', 'sugar_nu5_skewness']].hist(
    figsize=(12,6), bins=30, color='skyblue'
)
plt.suptitle("Distribution of Selected Structural Features")
plt.show()
```

Distribution of Selected Structural Features



```
#heatmap
import seaborn as sns

plt.figure(figsize=(12,10))
sns.heatmap(X_df.corr(), cmap="coolwarm", center=0)
plt.title("Correlation Heatmap of RNA Structural Features")
plt.show()
```


Correlation Heatmap of RNA Structural Features

```
clean_df = X_df.copy()
clean_df["Label"] = y

clean_df.to_csv("preprocessed_dataset.csv", index=False)
```

```
print(data.columns)
```



```
Index(['Motifname', 'Motifseq', 'bb_alpha_mean', 'bb_alpha_median',
      'bb_alpha_std', 'bb_alpha_range', 'bb_alpha_skewness',
      'bb_alpha_kurtosis', 'bb_beta_mean', 'bb_beta_median', 'bb_beta_std',
      'bb_beta_range', 'bb_beta_skewness', 'bb_beta_kurtosis',
      'bb_gamma_mean', 'bb_gamma_median', 'bb_gamma_std', 'bb_gamma_range',
      'bb_gamma_skewness', 'bb_gamma_kurtosis', 'bb_delta_mean',
      'bb_delta_median', 'bb_delta_std', 'bb_delta_range',
      'bb_delta_skewness', 'bb_delta_kurtosis', 'bb_eps_mean',
      'bb_eps_median', 'bb_eps_std', 'bb_eps_range', 'bb_eps_skewness',
      'bb_eps_kurtosis', 'bb_zeta_mean', 'bb_zeta_median', 'bb_zeta_std',
      'bb_zeta_range', 'bb_zeta_skewness', 'bb_zeta_kurtosis', 'bb_chi_mean',
      'bb_chi_median', 'bb_chi_std', 'bb_chi_range', 'bb_chi_skewness',
      'bb_chi_kurtosis', 'pucker_Phase_mean', 'pucker_Phase_median',
      'pucker_Phase_std', 'pucker_Phase_range', 'pucker_Phase_skewness',
      'pucker_Phase_kurtosis', 'pucker_amplitude_mean', 'pucker_amplitude_median',
      'pucker_amplitude_std', 'pucker_amplitude_range', 'pucker_amplitude_skewness',
      'pucker_amplitude_kurtosis', 'sugar_nu1_mean', 'sugar_nu1_median',
      'sugar_nu1_std', 'sugar_nu1_range', 'sugar_nu1_skewness',
      'sugar_nu1_kurtosis', 'sugar_nu2_mean', 'sugar_nu2_median',
      'sugar_nu2_std', 'sugar_nu2_range', 'sugar_nu2_skewness',
      'sugar_nu2_kurtosis', 'sugar_nu3_mean', 'sugar_nu3_median',
      'sugar_nu3_std', 'sugar_nu3_range', 'sugar_nu3_skewness',
      'sugar_nu3_kurtosis', 'sugar_nu4_mean', 'sugar_nu4_median',
      'sugar_nu4_std', 'sugar_nu4_range', 'sugar_nu4_skewness',
      'sugar_nu4_kurtosis', 'sugar_nu5_mean', 'sugar_nu5_median',
      'sugar_nu5_std', 'sugar_nu5_range', 'sugar_nu5_skewness',
      'sugar_nu5_kurtosis', 'Label'],
      dtype='object')
```

```
#Feature scaling
#clean dataset
data_clean = data.dropna()
```

```
#define terms x and y
X = data_clean.drop(columns=["Label", "Motifname", "Motifseq"])
y = data_clean["Label"]

print(X.shape)
```



```
print(y.shape)
```

```
(201375, 84)
(201375,)
```

```
#test/train split
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
#scale after split
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
#Multinomial logistic regression softmax
#1. Standardize data
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
from sklearn.linear_model import LogisticRegression

log_reg = LogisticRegression(
    solver='lbfgs',
    class_weight='balanced',
    max_iter=1000
)

log_reg.fit(X_train_scaled, y_train)
```

```
▼ LogisticRegression ⓘ ?
LogisticRegression(class_weight='balanced', max_iter=1000)
```

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

y_pred_lr = log_reg.predict(X_test_scaled)
```

```
print("Logistic Regression Accuracy:", accuracy_score(y_test, y_pred_lr))
print(classification_report(y_test, y_pred_lr))
```

Logistic Regression Accuracy: 0.5739044072004966

	precision	recall	f1-score	support
1x1-clips	0.36	0.63	0.46	1347
1x2-clips	0.26	0.43	0.33	986
1x3-clips	0.40	0.55	0.46	846
1x4-clips	0.42	0.49	0.45	543
1x5-clips	0.35	0.79	0.49	105
2x2-clips	0.33	0.44	0.38	724
2x3-clips	0.46	0.34	0.39	805
2x4-clips	0.50	0.58	0.53	344
2x5-clips	0.33	0.62	0.43	146
3x3-clips	0.40	0.29	0.34	692
3x4-clips	0.45	0.45	0.45	385
3x5-clips	0.07	0.62	0.13	21
4x4-clips	0.09	0.47	0.15	36
4x5-clips	0.25	0.61	0.35	84
5x5-clips	0.51	0.72	0.60	87
bulge1-clips	0.83	0.54	0.65	8650
bulge2-clips	0.45	0.39	0.42	3054
bulge3-clips	0.24	0.37	0.29	767
bulge4-clips	0.18	0.67	0.29	226
bulge5-clips	0.07	0.57	0.12	63
hairpin3-clips	0.26	0.82	0.40	798
hairpin4-clips	0.92	0.72	0.81	9733
hairpin5-clips	0.55	0.57	0.56	3923
hairpin6-clips	0.64	0.52	0.57	3241
hairpin7-clips	0.64	0.64	0.64	2669
accuracy			0.57	40275
macro avg	0.40	0.55	0.43	40275
weighted avg	0.66	0.57	0.60	40275

```
#train random forest
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=200,
    random_state=42,
    n_jobs=-1,
    class_weight='balanced'
)

rf.fit(X_train, y_train)
```

RandomForestClassifier

RandomForestClassifier(class_weight='balanced', n_estimators=200, n_jobs=-1, random_state=42)

```
#evaluate random forest
y_pred_rf = rf.predict(X_test)

print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))
print(classification_report(y_test, y_pred_rf))
```

```
Random Forest Accuracy: 0.9391185599006828
```

	precision	recall	f1-score	support
1x1-clips	0.94	0.89	0.91	1347
1x2-clips	0.91	0.90	0.90	986
1x3-clips	0.91	0.90	0.91	846
1x4-clips	0.95	0.86	0.90	543
1x5-clips	0.97	0.84	0.90	105
2x2-clips	0.93	0.87	0.90	724
2x3-clips	0.90	0.91	0.91	805
2x4-clips	0.94	0.85	0.89	344
2x5-clips	0.97	0.75	0.85	146
3x3-clips	0.91	0.91	0.91	692
3x4-clips	0.93	0.86	0.89	385
3x5-clips	1.00	0.76	0.86	21
4x4-clips	0.95	0.58	0.72	36
4x5-clips	0.96	0.87	0.91	84
5x5-clips	1.00	0.91	0.95	87
bulge1-clips	0.91	0.99	0.95	8650
bulge2-clips	0.91	0.93	0.92	3054
bulge3-clips	0.96	0.86	0.91	767
bulge4-clips	0.95	0.74	0.83	226
bulge5-clips	1.00	0.51	0.67	63
hairpin3-clips	0.96	0.88	0.92	798
hairpin4-clips	0.96	0.99	0.97	9733
hairpin5-clips	0.97	0.88	0.92	3923
hairpin6-clips	0.96	0.91	0.93	3241
hairpin7-clips	0.97	0.92	0.94	2669
accuracy			0.94	40275
macro avg	0.95	0.85	0.89	40275
weighted avg	0.94	0.94	0.94	40275

```
# compare both models
import pandas as pd

results = pd.DataFrame({
    "Model": ["Logistic Regression", "Random Forest"],
    "Accuracy": [
        accuracy_score(y_test, y_pred_lr),
        accuracy_score(y_test, y_pred_rf)
    ]
})

print(results)
```

	Model	Accuracy
0	Logistic Regression	0.573904
1	Random Forest	0.939119

```
#feature importance
import pandas as pd

feature_importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": rf.feature_importances_
}).sort_values(by="Importance", ascending=False)

print(feature_importance.head(15))
```

	Feature	Importance
29	bb_eps_kurtosis	0.057617
26	bb_eps_std	0.045967
71	sugar_nu3_kurtosis	0.043189
70	sugar_nu3_skewness	0.035383
28	bb_eps_skewness	0.034850
27	bb_eps_range	0.030351
77	sugar_nu4_kurtosis	0.029543
65	sugar_nu2_kurtosis	0.025473
76	sugar_nu4_skewness	0.025393
47	pucker_Phase_kurtosis	0.024571
23	bb_delta_kurtosis	0.022889
33	bb_zeta_range	0.020908
46	pucker_Phase_skewness	0.020492
24	bb_eps_mean	0.020182
32	bb_zeta_std	0.019762

```
X = data_clean.drop(columns=["Label", "Motifname", "Motifseq"])
y = data_clean["Label"]

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```
#svm
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
# train svm
from sklearn.svm import LinearSVC

svm = LinearSVC(C=1, max_iter=5000)

svm.fit(X_train_scaled, y_train)
```

LinearSVC  

```
LinearSVC(C=1, max_iter=5000)
```

```
y_pred_svm = svm.predict(X_test_scaled)
```

```
from sklearn.metrics import accuracy_score, classification_report

print("SVM Accuracy:", accuracy_score(y_test, y_pred_svm))
print(classification_report(y_test, y_pred_svm))
```

```
SVM Accuracy: 0.6029050279329609
```

	precision	recall	f1-score	support
1x1-clips	0.47	0.22	0.30	1368
1x2-clips	0.45	0.07	0.12	998
1x3-clips	0.59	0.37	0.46	852
1x4-clips	0.44	0.42	0.43	532
1x5-clips	0.67	0.73	0.70	107
2x2-clips	0.60	0.30	0.40	764
2x3-clips	0.47	0.39	0.43	790
2x4-clips	0.51	0.63	0.56	326
2x5-clips	0.59	0.39	0.47	144
3x3-clips	0.42	0.30	0.35	700
3x4-clips	0.50	0.33	0.40	393
3x5-clips	1.00	0.52	0.68	29
4x4-clips	0.33	0.03	0.06	30
4x5-clips	0.14	0.01	0.02	76
5x5-clips	0.76	0.67	0.71	84
bulge1-clips	0.57	0.94	0.71	8635
bulge2-clips	0.35	0.22	0.27	3033
bulge3-clips	0.25	0.02	0.04	768
bulge4-clips	0.36	0.17	0.24	206
bulge5-clips	0.00	0.00	0.00	70

hairpin3-clips	0.54	0.09	0.16	867
hairpin4-clips	0.75	0.91	0.82	9750
hairpin5-clips	0.55	0.44	0.49	4008
hairpin6-clips	0.65	0.35	0.46	3151
hairpin7-clips	0.54	0.55	0.55	2594
accuracy			0.60	40275
macro avg	0.50	0.36	0.39	40275
weighted avg	0.58	0.60	0.56	40275

```
#hyper parameter tuning
# take subset for faster tuning
X_small = X_train.sample(n=50000, random_state=42)
y_small = y_train.loc[X_small.index]
```

```
from sklearn.model_selection import RandomizedSearchCV, StratifiedKFold
from sklearn.ensemble import RandomForestClassifier
```

```
param_dist_rf = {
    'n_estimators': [100, 150, 200],
    'max_depth': [10, 20, None],
    'min_samples_leaf': [1, 2, 5],
    'min_samples_split': [2, 5]
}
```

```
cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)
```

```
rf = RandomForestClassifier(random_state=42, n_jobs=-1)
```

```
rf_search = RandomizedSearchCV(
    rf,
    param_distributions=param_dist_rf,
    n_iter=5, # keep small for speed
    scoring='f1_macro',
    cv=cv,
    verbose=2,
    n_jobs=-1,
    random_state=42
)
```

```
rf_search.fit(X_small, y_small)
```

```
print("Best RF Params:", rf_search.best_params_)
print("Best CV Score:", rf_search.best_score_)
```

```
Fitting 3 folds for each of 5 candidates, totalling 15 fits
Best RF Params: {'n_estimators': 150, 'min_samples_split': 2, 'min_samples_leaf': 5}
Best CV Score: 0.7877243263344065
```

```
#evaluation metrics
#accuracy
from sklearn.metrics import accuracy_score

print("Accuracy:", accuracy_score(y_test, y_pred_rf))
```

Accuracy: 0.1421725636250776

```
#PRECISION, RECALL, F1
from sklearn.metrics import precision_score, recall_score, f1_score

print("Precision (Macro):", precision_score(y_test, y_pred_rf, average='macro'))
print("Recall (Macro):", recall_score(y_test, y_pred_rf, average='macro'))
print("F1 Score (Macro):", f1_score(y_test, y_pred_rf, average='macro'))

print("F1 Score (Weighted):", f1_score(y_test, y_pred_rf, average='weighted'))
```

Precision (Macro): 0.040765537967775926
 Recall (Macro): 0.04029310230284207
 F1 Score (Macro): 0.04035576467623051
 F1 Score (Weighted): 0.13954923623679402

```
#full classification report
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred_rf))
```

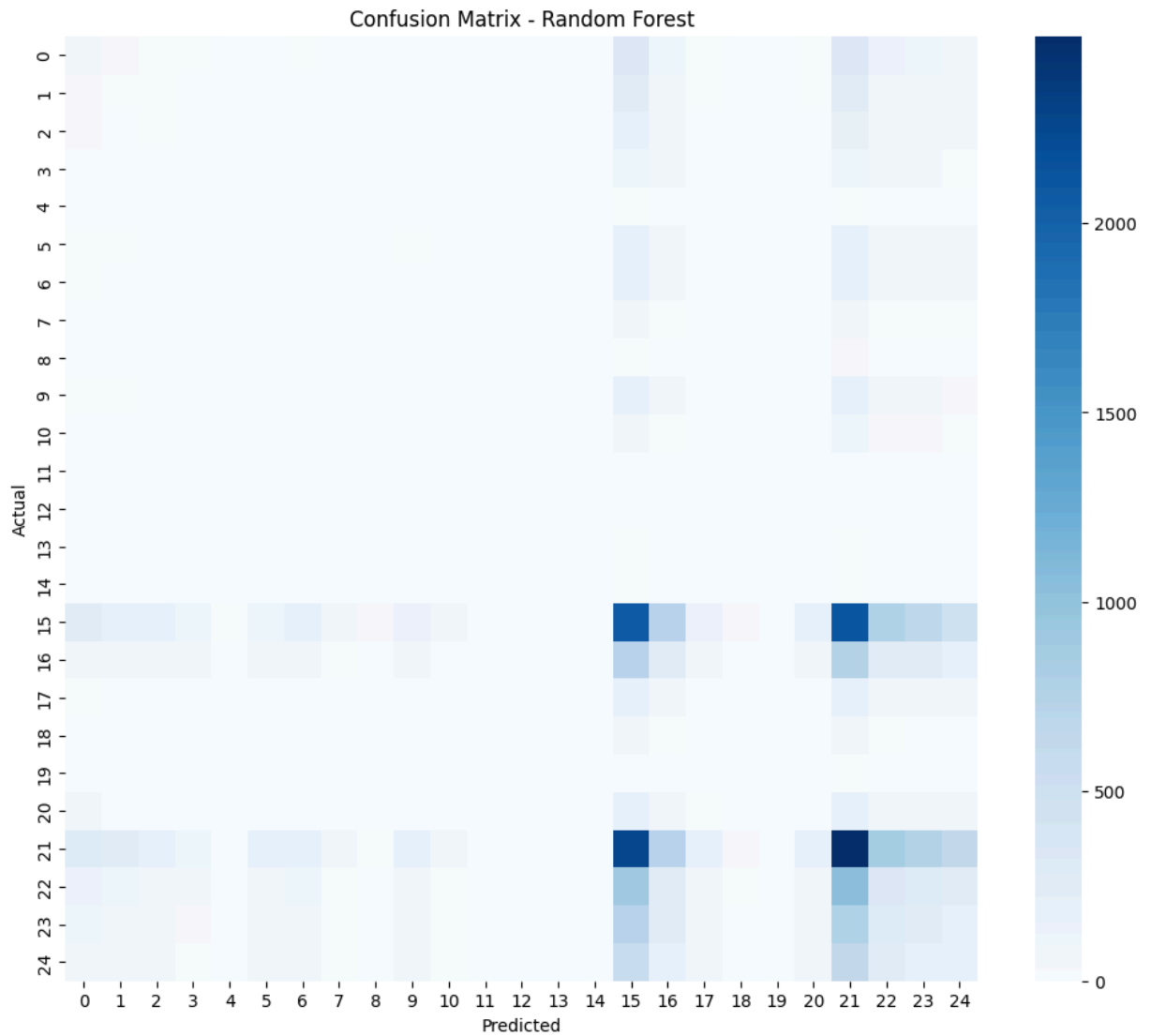
	precision	recall	f1-score	support
1x1-clips	0.03	0.03	0.03	1368
1x2-clips	0.03	0.03	0.03	998
1x3-clips	0.03	0.02	0.02	852
1x4-clips	0.01	0.01	0.01	532
1x5-clips	0.00	0.00	0.00	107
2x2-clips	0.02	0.02	0.02	764
2x3-clips	0.02	0.02	0.02	790
2x4-clips	0.00	0.00	0.00	326
2x5-clips	0.00	0.00	0.00	144
3x3-clips	0.02	0.02	0.02	700
3x4-clips	0.01	0.01	0.01	393
3x5-clips	0.00	0.00	0.00	29
4x4-clips	0.00	0.00	0.00	30
4x5-clips	0.00	0.00	0.00	76
5x5-clips	0.00	0.00	0.00	84
bulge1-clips	0.22	0.24	0.23	8635
bulge2-clips	0.08	0.08	0.08	3033
bulge3-clips	0.01	0.01	0.01	768
bulge4-clips	0.00	0.00	0.00	206
bulge5-clips	0.03	0.01	0.02	70
hairpin3-clips	0.02	0.02	0.02	867
hairpin4-clips	0.25	0.26	0.25	9750
hairpin5-clips	0.10	0.09	0.09	4008
hairpin6-clips	0.08	0.08	0.08	3151

hairpin7-clips	0.07	0.07	0.07	2594
accuracy			0.14	40275
macro avg	0.04	0.04	0.04	40275
weighted avg	0.14	0.14	0.14	40275

```
#confusion matrix
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred_rf)

plt.figure(figsize=(12,10))
sns.heatmap(cm, cmap="Blues")
plt.title("Confusion Matrix - Random Forest")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

```
#ROC VS AUC
from sklearn.preprocessing import label_binarize
```

```
classes = sorted(y.unique())
y_test_bin = label_binarize(y_test, classes=classes)
```

```
best_rf = rf_search.best_estimator_
y_score = best_rf.predict_proba(X_test)
```

```
from sklearn.metrics import roc_auc_score

roc_auc = roc_auc_score(y_test_bin, y_score, average='macro', multi_class='ovr')

print("ROC-AUC (Macro):", roc_auc)
```

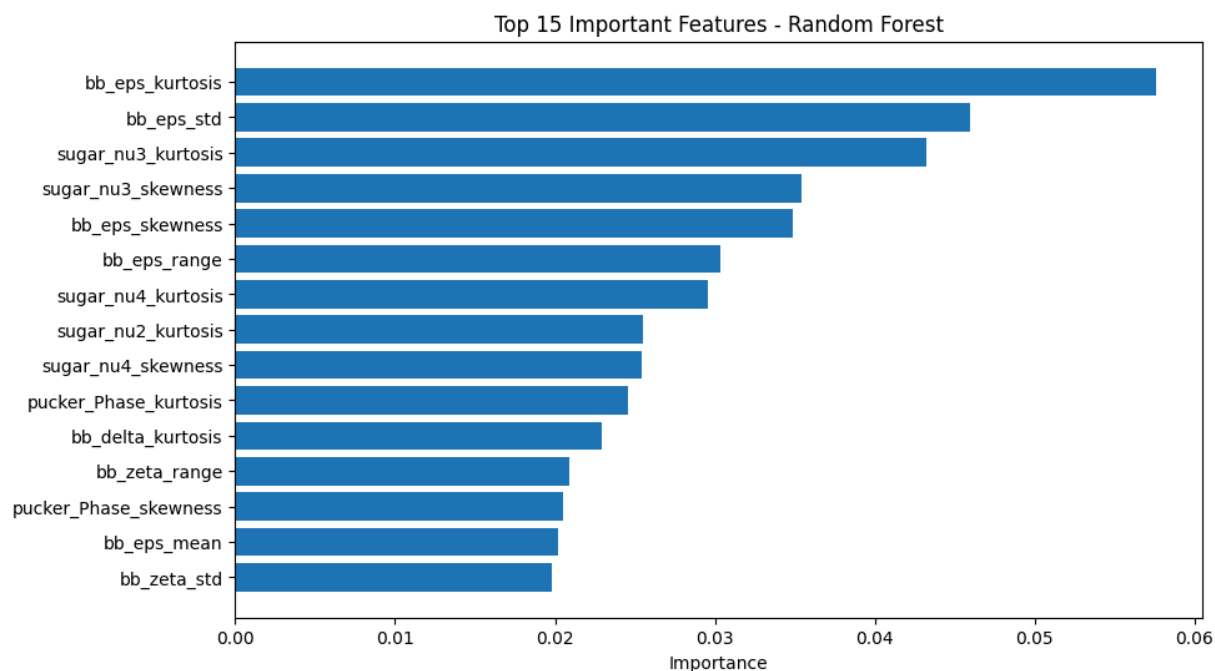
```
ROC-AUC (Macro): 0.9874464033909544
```

```
#learning curve random forest
from sklearn.model_selection import learning_curve
import numpy as np
import matplotlib.pyplot as plt

X_small = X_train.sample(n=30000, random_state=42)
y_small = y_train.loc[X_small.index]

train_sizes, train_scores, test_scores = learning_curve(
    rf,
    X_small,
    y_small,
    cv=3,
    scoring='accuracy',
    n_jobs=-1,
    train_sizes=np.linspace(0.1, 1.0, 5)
)
```

```
plt.figure(figsize=(10,6))
plt.barh(feature_importance["Feature"][:15][::-1],
         feature_importance["Importance"][:15][::-1])
plt.title("Top 15 Important Features - Random Forest")
plt.xlabel("Importance")
plt.show()
```



#PCA Dimensionality reduction

```
X = data.drop(columns=["Label", "Motifname", "Motifseq"])
y = data["Label"]

X = X.fillna(X.mean())

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

from sklearn.decomposition import PCA

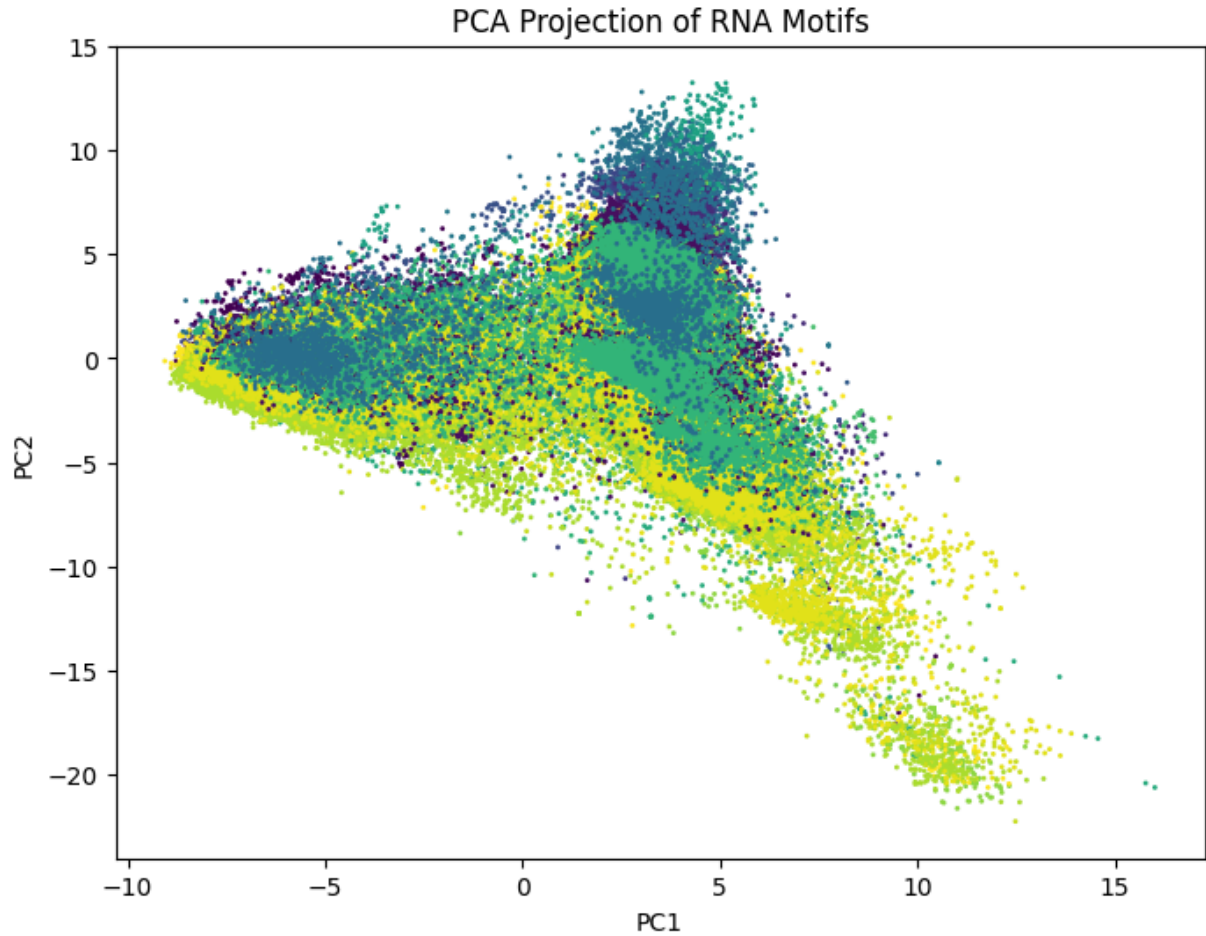
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

print("Explained variance:", pca.explained_variance_ratio_)

plt.figure(figsize=(8,6))
```

```
plt.scatter(X_pca[:,0], X_pca[:,1], c=y.astype('category').cat.codes, cmap='vir')
plt.title("PCA Projection of RNA Motifs")
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.show()
```

Explained variance: [0.2505308 0.14175658]



```
#web app for ml models
!pip install streamlit
```

```
Requirement already satisfied: streamlit in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: altair!=5.4.0,!5.4.1,<7,>=4.0 in /usr/local/lib/
Requirement already satisfied: blinker<2,>=1.5.0 in /usr/local/lib/python3.12/di
Requirement already satisfied: cachetools<8,>=5.5 in /usr/local/lib/python3.12/d
Requirement already satisfied: click<9,>=7.0 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: gitpython!=3.1.19,<4,>=3.0.7 in /usr/local/lib/py
Requirement already satisfied: numpy<3,>=1.23 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: packaging>=20 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: pandas<4,>=1.4.0 in /usr/local/lib/python3.12/dis
Requirement already satisfied: pillow<13,>=7.1.0 in /usr/local/lib/python3.12/di
```

```

Requirement already satisfied: pydeck<1,>=0.8.0b4 in /usr/local/lib/python3.12/d
Requirement already satisfied: protobuf<8,>=3.20 in /usr/local/lib/python3.12/di
Requirement already satisfied: pyarrow>=7.0 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: requests<3,>=2.27 in /usr/local/lib/python3.12/di
Requirement already satisfied: tenacity<10,>=8.1.0 in /usr/local/lib/python3.12/
Requirement already satisfied: toml<2,>=0.10.1 in /usr/local/lib/python3.12/dist
Requirement already satisfied: typing-extensions<5,>=4.10.0 in /usr/local/lib/py
Requirement already satisfied: starlette>=0.40.0 in /usr/local/lib/python3.12/di
Requirement already satisfied: uvicorn>=0.30.0 in /usr/local/lib/python3.12/dist
Requirement already satisfied: httptools>=0.6.3 in /usr/local/lib/python3.12/dis
Requirement already satisfied: anyio>=4.0.0 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: python-multipart>=0.0.10 in /usr/local/lib/python
Requirement already satisfied: websockets>=12.0.0 in /usr/local/lib/python3.12/d
Requirement already satisfied: itsdangerous>=2.1.2 in /usr/local/lib/python3.12/
Requirement already satisfied: watchdog<7,>=2.1.5 in /usr/local/lib/python3.12/d
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: jsonschema>=3.0 in /usr/local/lib/python3.12/dist
Requirement already satisfied: narwhals>=1.14.2 in /usr/local/lib/python3.12/dis
Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: gitdb<5,>=4.0.1 in /usr/local/lib/python3.12/dist
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/d
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/d
Requirement already satisfied: h11>=0.8 in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: smmap<6,>=3.0.1 in /usr/local/lib/python3.12/dist
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist
Requirement already satisfied: attrs>=22.2.0 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /usr/loca
Requirement already satisfied: referencing>=0.28.4 in /usr/local/lib/python3.12/
Requirement already satisfied: rpds-py>=0.25.0 in /usr/local/lib/python3.12/dist
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packag

```

```

import streamlit as st
import numpy as np
import pickle

st.title("RNA Motif Classification App")

# load model
model = pickle.load(open("rf_model.pkl", "rb"))

st.write("Enter feature values:")

# example input
features = []
for i in range(84):
    val = st.number_input(f"Feature {i}", value=0.0)
    features.append(val)

if st.button("Predict"):

```

```
prediction = model.predict([features])
st.write("Predicted Motif Class:" prediction[0])
```

```
2026-05-06 00:13:25.615 Thread 'MainThread': missing ScriptRunContext! This wa
2026-05-06 00:13:25.616 Thread 'MainThread': missing ScriptRunContext! This wa
2026-05-06 00:13:25.616 Thread 'MainThread': missing ScriptRunContext! This wa
2026-05-06 00:13:25.617 Thread 'MainThread': missing ScriptRunContext! This wa
2026-05-06 00:13:25.618 Thread 'MainThread': missing ScriptRunContext! This wa
2026-05-06 00:13:25.619 Thread 'MainThread': missing ScriptRunContext! This wa
2026-05-06 00:13:25.620 Thread 'MainThread': missing ScriptRunContext! This wa
2026-05-06 00:13:25.622 Thread 'MainThread': missing ScriptRunContext! This wa
2026-05-06 00:13:25.623 Thread 'MainThread': missing ScriptRunContext! This wa
2026-05-06 00:13:25.623 Thread 'MainThread': missing ScriptRunContext! This wa
2026-05-06 00:13:25.624 Thread 'MainThread': missing ScriptRunContext! This wa
2026-05-06 00:13:25.624 Thread 'MainThread': missing ScriptRunContext! This wa
2026-05-06 00:13:25.625 Thread 'MainThread': missing ScriptRunContext! This wa
2026-05-06 00:13:25.626 Thread 'MainThread': missing ScriptRunContext! This wa
2026-05-06 00:13:25.627 Thread 'MainThread': missing ScriptRunContext! This wa
2026-05-06 00:13:25.628 Thread 'MainThread': missing ScriptRunContext! This wa
2026-05-06 00:13:25.628 Thread 'MainThread': missing ScriptRunContext! This wa
```